

@inventory_monitor

Appears in: **DL-02**

orchestra/agents/inventory_monitor.py:18-83

"Orchestra runs three specialist agents in parallel — inventory_monitor, supplier_scout, and invoice_auditor (@inventory_monitor · @supplier_scout · @invoice_auditor) ..."

@inventory_monitor

```
orchestra/agents/inventory_monitor.py:18-83

18 def run(brief: dict, llm=None) -> list:
19     """Returns list of Finding dicts. Callable standalone."""
20     if llm is None:
21         llm = get_llm()
22     inventory = brief["inventory"]
23     open_pos = brief.get("open_pos", [])
24     open_skus = {po["sku_id"] for po in open_pos}
25
26     findings = []
27     for item in inventory:
28         if item["current_stock"] >= item["reorder_point"]:
29             continue
30         if item["sku_id"] in open_skus:
31             continue # PO already pending
32
33         daily = max(item.get("daily_usage_avg", 1), 0.1)
34         days = round(item["current_stock"] / daily, 1)
35
36         result = call_json(
37             llm,
38             system=(
39                 "You are a warehouse analyst. Return ONLY valid JSON -- no markdown:\n"
40                 '{"urgency": "critical|high|medium|low", "recommended_qty": int, '
41                 '"confidence": 0.0-1.0, "reasoning": "..."}'
42             ),
43             user=(
44                 f"SKU: {item['sku_id']} -- {item['name']}\n"
45                 f"Current stock: {item['current_stock']} | Reorder point: {item['reorder_point']} | "
46                 f"Max stock: {item['max_stock']}\n"
47                 f"Daily usage: {daily} | Days remaining: {days}\n"
48                 f"Open POs for this SKU: {item['sku_id'] in open_skus}"
49             ),
50         )
51
52         if result.get("fallback"):
53             urgency = "critical" if days < 2.0 else "high" if days < 5.0 else "medium"
54             rec_qty = item["max_stock"] - item["current_stock"]
55             confidence = min(1.0 / max(days, 0.1) / 10, 1.0)
56             reasoning = f"Rule-based: {days} days remaining"
57         else:
58             urgency = result.get("urgency", "high")
59             rec_qty = int(result.get("recommended_qty", item["max_stock"] - item["current_stock"]))
60             confidence = float(result.get("confidence", 0.8))
61             reasoning = result.get("reasoning", "")
62
63         findings.append({
64             "agent": AGENT_NAME,
65             "type": "stockout_risk",
66             "severity": "critical" if urgency == "critical" else "warning",
67             "sku_id": item["sku_id"],
68             "sku_name": item["name"],
69             "urgency": urgency,
70             "confidence": confidence,
71             "recommended_qty": rec_qty,
72             "data": {
73                 "current_stock": item["current_stock"],
74                 "reorder_point": item["reorder_point"],
75                 "max_stock": item["max_stock"],
76                 "days_remaining": days,
77                 "unit_cost_avg": item.get("unit_cost_avg", 0),
78             },
79             "reasoning": reasoning,
80             "recommendation": "generate_po",
81         })
82
83     return findings
```

What it shows: Specialist agent #1: scans inventory, emits stockout_risk findings with urgency/confidence. Read-only (works from the shared brief).

Source: orchestra/agents/inventory_monitor.py:18-83 · image rendered by build_snippets.py